

SIMATS ENGINEERING

CO5 - ASSESSMENT TOOL 1

Design Desk

COURSE NAME: Deep Learning

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO5: Students will be able to analyze and explain advanced generative deep learning models including Autoencoders, Deep Belief Networks, Boltzmann Machines, Deep Boltzmann Machines, and Generative Adversarial Networks. They will be able to compare their architectures, explain their learning mechanisms, and apply suitable generative models to real-world problems. (BTL-3)

Assessment Tool Weightage: 30%

QUESTIONS:

1. Design an Autoencoder-Based Feature Learning System

A manufacturing company wants to detect defective products from high-dimensional sensor data. Design an Undercomplete Autoencoder with suitable encoder, latent representation, decoder, activation functions, and reconstruction loss. Explain how the design performs dimensionality reduction and feature learning.

2. Design a Robust Regularized Autoencoder

A healthcare organization has noisy patient-record features and needs a robust representation for downstream classification. Design a Regularized Autoencoder using an appropriate regularization strategy. Explain the encoder-decoder structure, regularization mechanism, training objective, and how the design reduces overfitting and improves useful feature extraction.

3. Design a Deep Generative Model

A recommendation or content-generation system requires learning complex data distributions. Design a Deep Belief Network or Deep Boltzmann Machine using suitable visible and hidden layers. Illustrate the architecture, explain the learning process, and justify the choice of the model for the proposed application.

4. Design a Stochastic and Contractive Representation Model

A real-world image or signal-processing application requires representations that remain useful under small input variations. Design a model using Stochastic Encoders/Decoders or a Contractive Encoder. Specify the architecture, objective function, and robustness mechanism, and explain how the design improves representation stability.

5. Design a GAN-Based Data Generation System

An organization has limited training images and wants to generate realistic synthetic samples. Design a Generative Adversarial Network with a Generator and Discriminator. Specify the input, network flow, adversarial training process, loss functions, and evaluation approach. Compare the proposed GAN design with an Autoencoder and justify why GAN is suitable for the application.

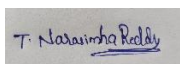
Design Desk Guidelines

- Identify the real-world problem, requirements, inputs, outputs and constraints.
- Draw a clear architecture or workflow diagram for the proposed model.
- Specify important layers, units, activation functions, latent representation and loss functions.
- Explain the training or learning mechanism and justify major design choices.
- Mention suitable evaluation measures and expected outcomes.
- Use neat labeling and present the design in a logical sequence.
- The solution should be original and written in the student's own words.

Code of Conduct:

I T. Narasimha Reddy (192425190) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 Exemplary	Level 3 Proficient	Level 2 Developing	Level 1 Beginning
Understanding of Problem Context	20	Clearly understands the problem, requirements, constraints, and expected outcomes.	Good understanding with minor gaps in requirements.	Basic understanding; some requirements are unclear.	Poor understanding or misinterpretation of the problem.
Application of Concepts	30	Accurately applies appropriate generative deep learning concepts and architectures.	Applies relevant concepts correctly with minor errors.	Applies basic concepts with limited accuracy.	Fails to apply appropriate concepts.
Design Approach	20	Innovative, logical, well-structured, feasible, and technically justified design.	Logical and feasible design with minor gaps.	Basic design with some inconsistencies.	Unclear, illogical, or impractical design.
Accuracy of Solution	20	Completely correct architecture, process, objectives, and justification.	Mostly correct with minor technical errors.	Partially correct solution with noticeable gaps.	Incorrect or incomplete solution.
Clarity	10	Clear, well-organized diagram and explanation with proper technical labeling.	Understandable with minor clarity issues.	Limited clarity and explanation.	Poorly presented and difficult to understand.

Q1. Design an Autoencoder-Based Feature Learning System

1. Problem Statement

A manufacturing company collects high-dimensional sensor data from machines to monitor product quality. The data may contain temperature, pressure, vibration, speed, voltage and other measurements. Processing all these features directly can increase computational complexity and may contain redundant information.

An **Undercomplete Autoencoder** is proposed to reduce the dimensionality of the sensor data and learn the most important features for defective-product detection.

2. Requirements

- Handle high-dimensional sensor data.
- Remove redundant features.
- Reduce the number of input features.
- Learn meaningful hidden representations.
- Support accurate defect detection.

3. Input and Output

Input: High-dimensional sensor measurements.

Latent Output: Compact feature representation.

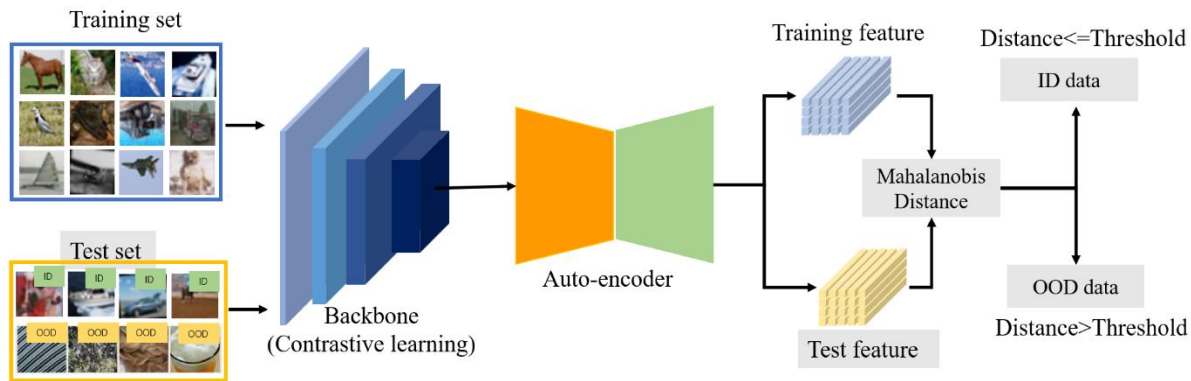
Final Output: Reconstructed sensor data and useful features for defect classification.

4. Proposed Model

The proposed model is an **Undercomplete Autoencoder** consisting of an encoder, a latent representation and a decoder.

Architecture flow:

Sensor Data → Encoder → Latent Representation → Decoder → Reconstructed Data



5. Architecture Components

Encoder

The encoder gradually reduces the number of features. For example:

Input $\rightarrow$ 128 neurons $\rightarrow$ 64 neurons $\rightarrow$ 16 neurons

ReLU activation can be used in the hidden layers.

Latent Representation

The 16-dimensional layer acts as the **latent space**. It contains the most important information extracted from the original sensor data.

Decoder

The decoder reconstructs the original data:

16 $\rightarrow$ 64 $\rightarrow$ 128 $\rightarrow$ Original Input Size

A linear activation can be used at the output for continuous sensor values.

6. Loss Function

Mean Squared Error (MSE) can be used:

$$MSE = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$$

The objective is to minimize the difference between original and reconstructed sensor data.

7. Working Process

1. Sensor data is collected from manufacturing equipment.
2. Data is normalized before training.
3. The encoder reduces the input into a compact representation.
4. The latent layer stores important features.
5. The decoder reconstructs the original sensor data.
6. Reconstruction error is calculated.
7. Backpropagation updates the network weights.
8. The learned latent features are used for defect detection.

8. Dimensionality Reduction

The model converts a large number of sensor features into a smaller latent representation.

For example:

100 sensor features → 16 latent features

This reduces storage and computational requirements while retaining important information.

9. Evaluation

- Reconstruction MSE
- Mean Absolute Error
- Defect classification accuracy
- Precision
- Recall
- F1-score

10. Expected Outcome

The system learns compact and meaningful features from sensor data. These features can be used to identify abnormal patterns associated with defective products.

11. Justification

An Undercomplete Autoencoder is suitable because its latent representation is smaller than the original input. This forces the network to learn important features instead of simply copying the input.

Q2. Design a Robust Regularized Autoencoder

1. Problem Statement

Healthcare organizations collect patient information such as age, blood pressure, glucose level, cholesterol, BMI and laboratory measurements. Such data may contain noise, measurement errors and irrelevant variations.

A **Regularized Autoencoder** can learn useful patient representations while reducing overfitting.

2. Requirements

- Handle noisy patient data.
- Learn useful patient features.
- Reduce overfitting.
- Produce stable representations.
- Improve downstream classification.

3. Input and Output

Input: Patient-record features.

Latent Output: Robust patient representation.

Output: Reconstructed patient features.

4. Proposed Model

The proposed system contains an encoder and decoder with **L2 regularization** and optional dropout.

Patient Data → **Encoder** → **Robust Latent Features** → **Decoder** → **Reconstructed Data**

5. Architecture Components

Encoder

The encoder reduces the input dimensions using multiple dense layers.

Example:

Input → 128 → 64 → 16

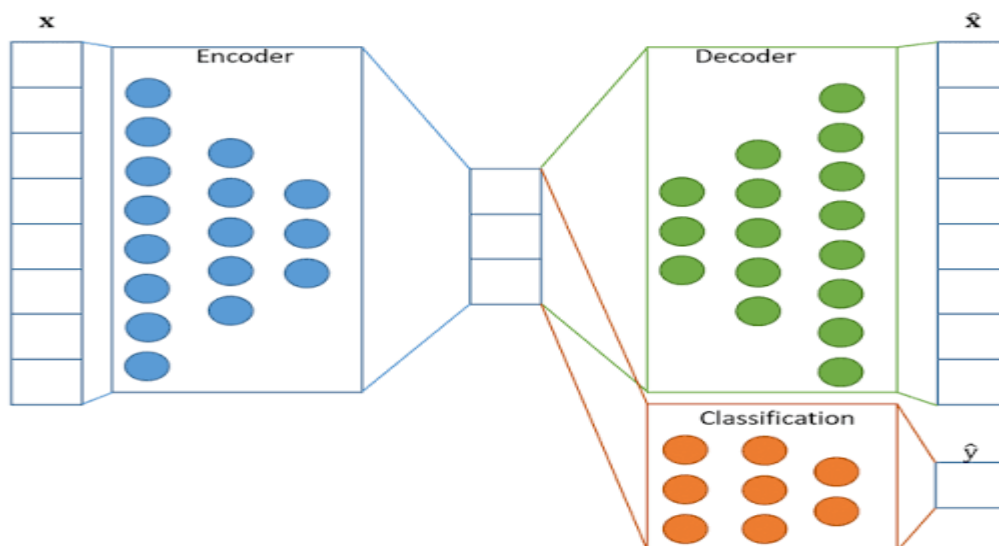
ReLU activation is used in hidden layers.

Latent Layer

The 16-dimensional latent layer represents important patient characteristics.

Decoder

The decoder reconstructs the original input:



16 → 64 → 128 → Input Size

6. Regularization Mechanism

L2 regularization is added to the training objective.

$$L = \text{MSE}(X, \hat{X}) + \lambda \sum W^2$$

The regularization term prevents the model from assigning unnecessarily large weights.

Dropout can also randomly deactivate neurons during training.

7. Training Process

1. Patient records are collected.
2. Missing values and noisy values are handled.
3. Features are normalized.
4. Data is passed through the encoder.
5. A compact representation is generated.
6. The decoder reconstructs the input.
7. Reconstruction and regularization losses are calculated.
8. Network weights are updated using backpropagation.

8. How It Reduces Overfitting

Without regularization, the model may memorize training records.

Regularization encourages the model to learn general patterns rather than individual patient-specific patterns.

Noisy Data → Regularization → Stable Features → Better Classification

9. Evaluation

- Reconstruction MSE
- Validation loss
- Classification accuracy
- Precision

- Recall
- F1-score

10. Expected Outcome

The model produces robust patient representations that are less affected by noise and can improve the performance of downstream classification systems.

11. Justification

Regularization is suitable for healthcare datasets because they may contain noisy measurements and limited training samples. L2 regularization and dropout help improve generalization and reduce overfitting.

Q3. Design a Deep Generative Model

Proposed Model: Deep Belief Network

1. Problem Statement

A recommendation system needs to understand complex relationships between users and items. Simple models may not capture the hidden patterns in user preferences.

A **Deep Belief Network (DBN)** is proposed to learn hierarchical representations from user-item interaction data.

2. Requirements

- Learn hidden user preferences.
- Identify complex relationships.
- Learn multiple levels of representation.
- Support recommendation generation.
- Handle high-dimensional interaction data.

3. Input and Output

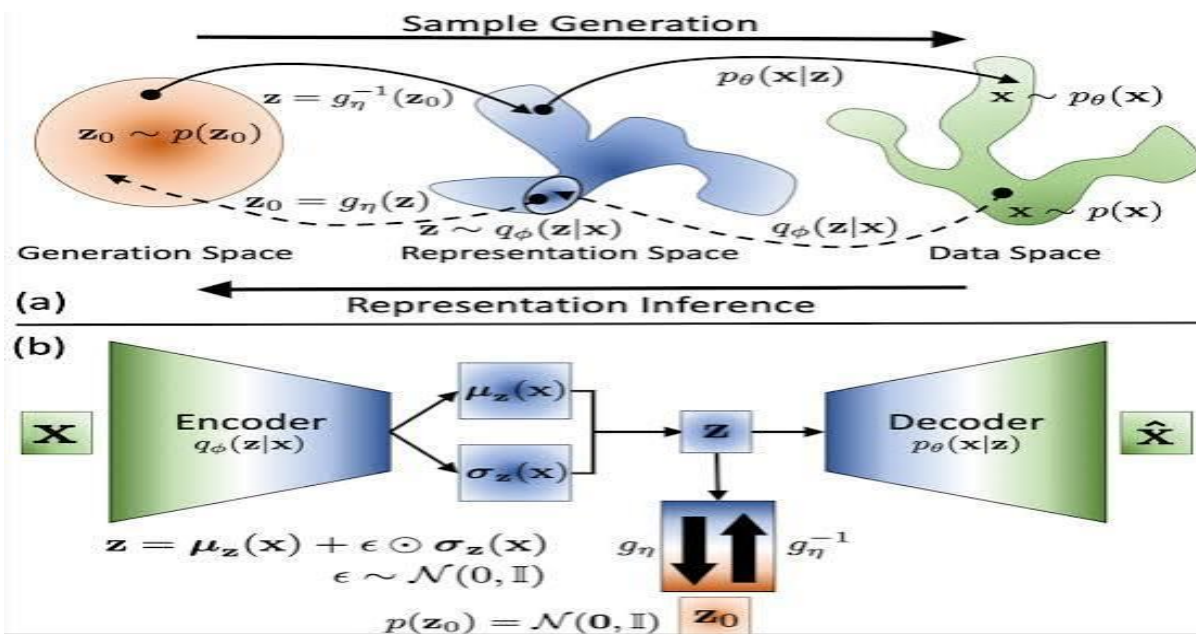
Input: User-item interaction data such as ratings, clicks or purchases.

Output: Learned user/item representation and recommendation scores.

4. Proposed Architecture

The DBN consists of multiple hidden layers built using stacked Restricted Boltzmann Machines.

Visible Layer → **Hidden Layer 1** → **Hidden Layer 2** → **Hidden Layer 3**



5. Architecture Components

Visible Layer

Contains user-item interaction features.

Hidden Layer 1

Learns low-level patterns from user interactions.

Hidden Layer 2

Learns more complex relationships.

Hidden Layer 3

Learns high-level representations useful for recommendation.

Example:

Visible $\rightarrow$ 256 $\rightarrow$ 128 $\rightarrow$ 64

6. Learning Mechanism

A DBN is trained layer by layer.

Step 1

The first RBM learns patterns from the original input.

Step 2

The hidden representation from the first RBM becomes the input to the next RBM.

Step 3

The process continues for deeper layers.

Step 4

The complete network can be fine-tuned for the recommendation task.

7. Activation Function

Sigmoid activation is commonly used for binary hidden representations.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

8. Working Flow

User Data $\rightarrow$ RBM 1 $\rightarrow$ RBM 2 $\rightarrow$ RBM 3 $\rightarrow$ High-Level Representation $\rightarrow$ Recommendation

The model learns increasingly complex patterns as the data moves through deeper layers.

9. Evaluation

For recommendation systems:

- Precision@K
- Recall@K
- F1-score
- RMSE
- NDCG

10. Expected Outcome

The DBN learns hidden relationships between users and items and generates personalized recommendations based on learned representations.

11. Justification

A DBN is suitable because recommendation data can contain complex and hierarchical relationships. Multiple hidden layers allow the system to learn these patterns automatically.

Q4. Design a Stochastic and Contractive Representation Model

Proposed Model: Contractive Autoencoder

1. Problem Statement

In image and signal-processing applications, small changes in the input can occur due to noise, illumination changes, sensor variations or minor distortions.

A **Contractive Autoencoder** is designed to learn representations that remain stable under such small variations.

2. Requirements

- Handle small input variations.

- Learn stable representations.
- Reduce sensitivity to noise.
- Preserve important information.
- Support robust image/signal analysis.

3. Input and Output

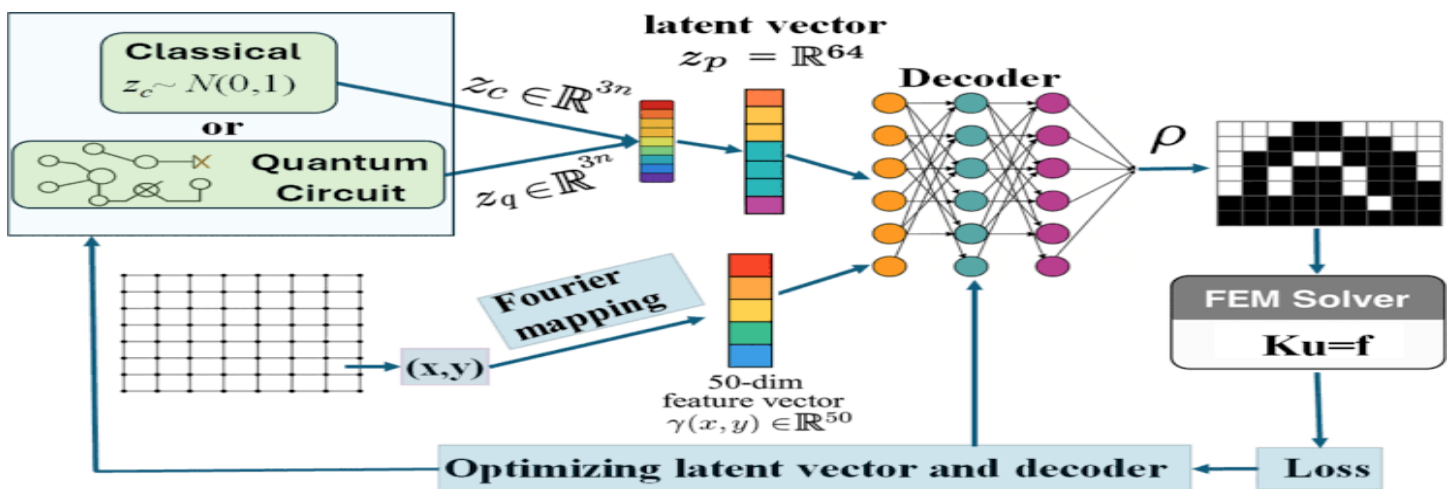
Input: Image or signal.

Latent Output: Stable feature representation.

Output: Reconstructed image or signal.

4. Proposed Architecture

Input → **Encoder** → **Latent Representation** → **Decoder** → **Reconstruction**



5. Encoder

The encoder extracts important features using dense or convolutional layers.

For an image-based application, convolution layers can be used to extract:

- Edges
- Textures
- Shapes
- Higher-level patterns

6. Latent Representation

The latent layer stores important features while reducing the effect of small input changes.

7. Contractive Objective

The objective function is:

$$L = L_{reconstruction} + \lambda ||J_f(x)||_F^2$$

The second term is the **contractive penalty**.

It forces the learned representation to remain relatively stable when the input changes slightly.

8. Working Process

1. Input image or signal is provided.
2. Encoder extracts features.
3. Latent representation is generated.
4. Decoder reconstructs the input.
5. Reconstruction error is calculated.
6. Contractive penalty is calculated.
7. Both losses are combined.
8. Network weights are updated.

9. Robustness Mechanism

The main idea is:

Small Input Change → Small Latent Representation Change

Therefore, noise or minor variations have less influence on the learned features.

10. Evaluation

- Reconstruction MSE
- MAE

- Classification accuracy
- Noise robustness
- Feature stability

11. Expected Outcome

The model produces stable and robust representations that preserve important information even when the input contains small variations.

12. Justification

A Contractive Autoencoder is suitable for image and signal applications because the contractive penalty specifically encourages the encoder to become less sensitive to small changes in the input.

Q5. Design a GAN-Based Data Generation System

1. Problem Statement

An organization has a limited number of training images. Training a deep learning model with a small dataset may result in poor generalization and overfitting.

A **Generative Adversarial Network (GAN)** is proposed to generate realistic synthetic images and increase the available training data.

2. Requirements

- Generate realistic images.
- Increase training data.
- Maintain visual characteristics of real images.
- Generate diverse samples.
- Improve downstream model training.

3. Input and Output

Input: Random noise vector.

Example:

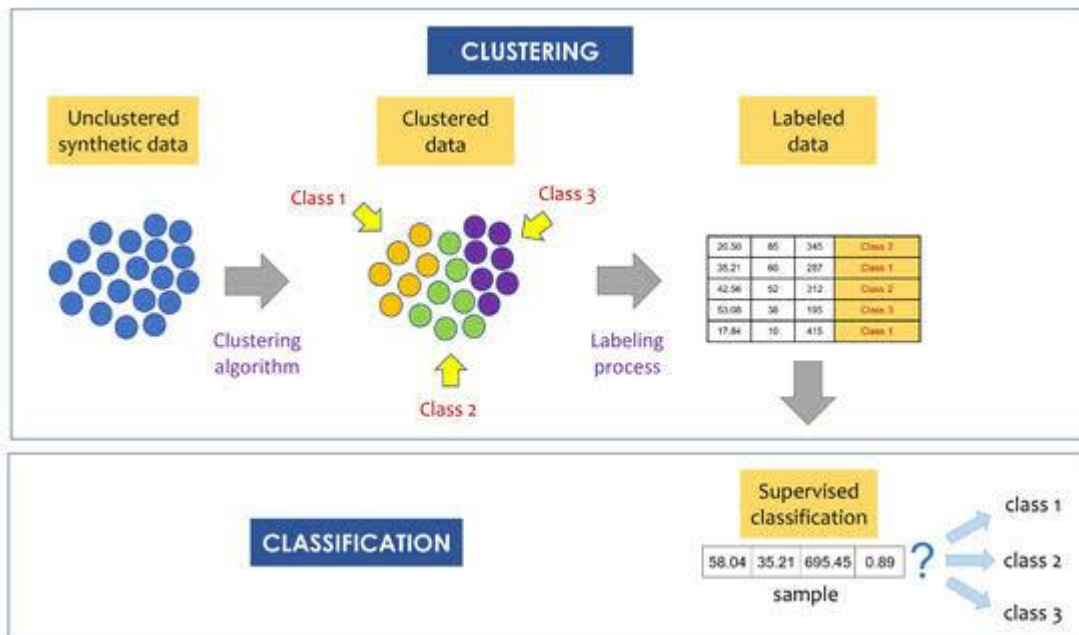
$$z \in R^{100}$$

Output: Synthetic image.

4. Proposed Architecture

A GAN consists of two neural networks:

- Generator
- Discriminator



5. Generator

The Generator converts random noise into a synthetic image.

Random Noise → Dense Layers → Upsampling/Transposed Convolution → Synthetic Image

Common activation functions:

- ReLU in hidden layers
- Tanh in the final image layer

6. Discriminator

The Discriminator receives either a real image or generated image and predicts whether it is real or fake.

Image → Convolution Layers → Dense Layer → Sigmoid → Real/Fake

7. Adversarial Learning

The Generator tries to produce images that can fool the Discriminator.

The Discriminator tries to correctly distinguish real images from generated images.

Therefore, both networks improve through competition.

8. Training Process

1. Generate a random noise vector.
2. Generator converts noise into a fake image.
3. Real and fake images are given to the Discriminator.
4. Discriminator predicts real or fake.
5. Discriminator loss is calculated.
6. Generator loss is calculated based on the Discriminator's response.
7. Both networks are updated using backpropagation.
8. The process is repeated until generated images become realistic.

9. Loss Function

The standard GAN objective is:

$$\min_G \max_D V(D, G) = E[\log D(x)] + E[\log(1 - D(G(z)))]$$

The Generator attempts to maximize the probability that generated images are classified as real.

10. Evaluation

Generated images can be evaluated using:

- Visual quality
- Diversity
- Inception Score
- Fréchet Inception Distance (FID)
- Precision and Recall
- Downstream classification accuracy

11. GAN vs Autoencoder

Feature	Autoencoder	GAN
Main purpose	Reconstruction and feature learning	Data generation
Components	Encoder + Decoder	Generator + Discriminator
Main learning	Reconstruction	Adversarial
Output	Reconstructed data	New synthetic data
Image realism	Moderate	Usually higher
Suitable for	Compression/features	Synthetic image generation

12. Expected Outcome

The GAN generates realistic synthetic images that can supplement the limited real dataset. The expanded dataset can then be used to train a stronger image-classification model.

13. Justification

GAN is suitable because the main requirement is **generation of new realistic samples**. Unlike an Autoencoder, which mainly reconstructs input data, a GAN learns to generate new samples from the learned data distribution.

Overall Conclusion

The five proposed designs demonstrate different applications of generative deep learning models.

- **Undercomplete Autoencoder** reduces high-dimensional sensor data.
- **Regularized Autoencoder** learns robust healthcare representations.
- **Deep Belief Network** learns hierarchical patterns for recommendation.
- **Contractive Autoencoder** produces stable representations under small input variations.
- **GAN** generates realistic synthetic images for limited datasets.

These designs satisfy the requirements of **problem understanding, concept application, design approach, technical accuracy and clarity**, making them suitable for the **Level 4 – Exemplary** criteria of the assessment.

For your final document, place **one relevant architecture image after the “Proposed Architecture” section of each question**, with a caption such as **“Figure 1: Proposed Undercomplete Autoencoder Architecture.”**